

ProcessPilot AI ? From Problem to Production

A Full-Stack AI Engineering & Problem-Solving Case Study

Problem -> Requirements -> Architecture -> Implementation -> Debugging -> Iteration -> Testing -> Deployment

Author: Aritra Das

Version: 1.0

Date: September 2026

1. Introduction: The Starting Point

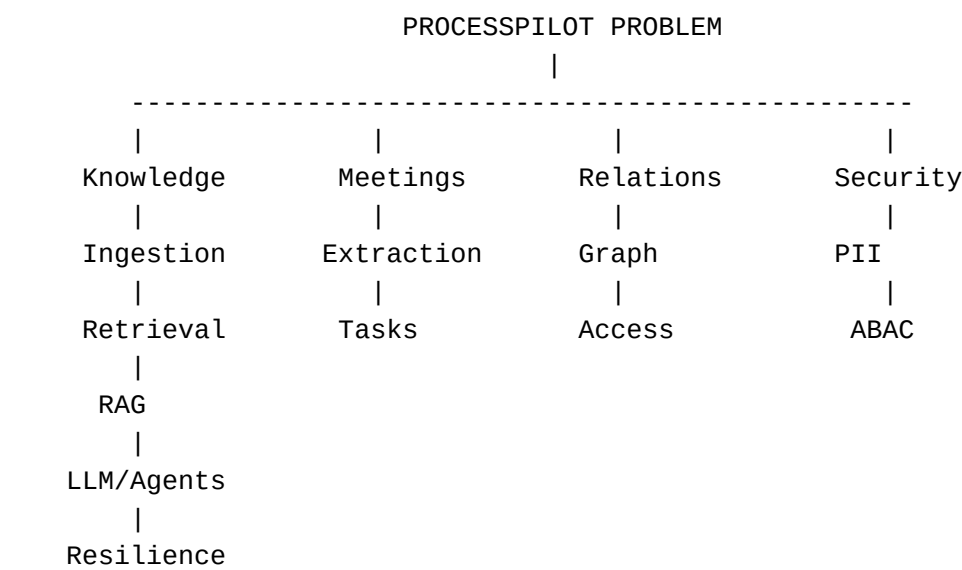
I started with a problem, not a stack.

Organizational knowledge is fundamentally fragmented. Documents contain policies, meetings produce action items, and HR systems define operational relationships. The problem I wanted to explore was: why do these silos remain isolated? Combining documents, meetings, tasks, and relationships creates a difficult engineering problem because relational data (SQL), semantic data (vectors), and conversational data (LLM outputs) require entirely different processing paradigms.

The challenge was not simply generating answers with an LLM. The challenge was building the surrounding system required to provide the right information, enforce access boundaries, handle failures, persist state, and operate under deployment constraints.

2. Problem Decomposition

To tackle this, I broke the massive problem into smaller, isolated engineering questions.



This visual tree directly influenced the architecture. It meant I could not just build a monolithic chatbot. I needed an ingestion engine for knowledge, a structured extraction engine for meetings, a graph layer for relations, and a security boundary overarching it all. The problem went from "ambiguous AI project" to specific micro-challenges.

3. PRD: Why Planning Came Before Code

Starting Uncertainty

What exactly was I building? A chat interface? A task manager? A document vault?

Risk

Starting development immediately would have led to uncontrolled feature growth. I would have built features looking for problems.

Decision

Write a Product Requirements Document (PRD) to establish users, workflows, and strict boundaries.

What the PRD Established

The PRD defined roles (Admins, Managers, Employees) and mapped user problems to technical

implications:

- Problem: Fragmented documents -> Requirement: Centralized ingestion -> Tech Implication: Vector retrieval architecture.
- Problem: Lost meeting actions -> Requirement: Transcript extraction -> Tech Implication: LLM structured output parsing.
- Problem: Relational queries -> Requirement: Organizational hierarchy -> Tech Implication: Knowledge Graph.

4. PRD to TRD: Translation from Product to Engineering

I did not choose technologies first. The requirements created technical constraints, and those constraints influenced my technology choices.

Requirement | Constraint | Technical Decision | Reason

RBAC / ABAC | Persistent relational connections | PostgreSQL | Users, departments, and permissions fit naturally into relational schemas.

Vector retrieval | Vector similarity search | pgvector | Allows relational (RBAC) and vector data to coexist in one database.

Real-time tasks | Server push to client | WebSockets (ws.py) | Avoids expensive HTTP polling when tasks are generated.

AI responses | LLM integration flexibility | Unified LLM client | Needed fallback logic to handle provider outages dynamically.

5. Architecture Design & Evolution

The architecture needed to support real-time chat streaming, synchronous document ingestion, and relational queries simultaneously.

Initial Architecture

React frontend, FastAPI backend, local HuggingFace embeddings, ChromaDB for vectors, and NetworkX JSON for the graph.

Problems Discovered

Deploying on a free-tier serverless environment (Render) exposed fatal flaws in the initial design. Ephemeral disks wiped the ChromaDB and NetworkX JSON files on every restart. Local embeddings consumed too much RAM.

Current Architecture

React 19 Frontend -> FastAPI Backend -> PostgreSQL (pgvector + relational tables). The backend acts as a stateless conduit. Embeddings shifted to a Gemini / Deterministic Mock setup to save memory.

6. Database Design: How the Data Model Emerged

I first asked what information the system needed to remember, rather than blindly creating tables.

- Users -> need identity and role.
- Departments -> define organizational access boundaries (ABAC).
- Documents -> represent uploaded knowledge.
- Chunks -> represent retrievable pieces of documents.
- Embeddings -> enable semantic retrieval.

- Meetings -> represent conversations.
- Tasks -> represent actionable items extracted from meetings.
- Agent Logs -> make AI routing behavior observable.
- LLM Usage -> track model token consumption/cost.
- AI Failures -> record provider timeouts that HTTP 200s mask.
- KG Nodes/Edges -> represent explicit graph relationships.

This reasoning culminated in an 18-table SQLAlchemy schema. The access boundaries are enforced directly via Foreign Keys linking documents and users to specific department_ids.

7. Document Ingestion: Deep Case Study

The Problem

How can arbitrary organizational documents become searchable AI knowledge?

Pipeline Design

File Validation -> PyMuPDF Extraction -> PII Redaction -> Semantic Chunking -> Embedding -> pgvector Storage

Why Each Stage Exists

Extraction converts PDFs to text. Redaction prevents data leaks. Chunking (with overlap) ensures the LLM receives context without exceeding token limits. Embedding translates text to vector math.

Problem Discovered

Initially, I processed files asynchronously. However, free-tier ephemeral filesystems deleted the uploaded PDF before the background thread could finish chunking it.

Solution & Trade-off

I forced the extraction to happen in-memory immediately upon upload. The trade-off is slower HTTP response times for massive PDFs, but it guarantees data persistence into Postgres before the ephemeral file is lost.

8. PII: Security as a Design Decision

The Problem

HR documents contain SSNs, credit cards, and sensitive information. Sending this raw content to an external LLM (OpenAI/Groq) creates a massive security exposure.

Design Decision

Redaction must occur **before** vectorization, not during retrieval.

Implementation & Trade-off

I implemented pii_redactor.py using Regex patterns. I considered heavy NLP (like Presidio), but free-tier RAM constraints made running a secondary ML model impossible. Regex is extremely fast but trades off by occasionally missing edge-case formats.

9. RAG: Showing the Evolution

The problem was not "How do I use RAG?". It was "How do I make an LLM answer questions using internal documents reliably?"

First Retrieval Approach

Dense semantic retrieval using pgvector cosine similarity ($\cos$).

Observation & Problem

It answered "How do I request time off?" perfectly. It completely failed on "Summarize Document ID HR-992-B". Dense vectors compress tokens into semantic space, destroying exact character matching.

Iteration

I added BM25Okapi to provide a sparse keyword retrieval signal.

Current Design & Limitation

The system uses Hybrid Retrieval. However, the scores are currently combined loosely. Reciprocal Rank Fusion (RRF) is mathematically superior for hybrid merging, and it is explicitly PLANNED but NOT IMPLEMENTED in the current iteration.

10. Embeddings: Constraint-Driven Decisions

To generate embeddings, I initially planned to use a local HuggingFace MiniLM model. However, deployment memory constraints on Render made hosting a local ML model unstable. I pivoted to using Gemini (google.generativeai) for production, but to save API costs during rapid development, I built a deterministic hash-based mock generator (768 dimensions).

11. Multi-Agent Architecture

The Problem

A standard RAG pipeline fails if a user asks, "Who is the manager of Engineering?". Vector search looks for semantic similarity, not SQL relationships.

Architectural Decision

I introduced controlled routing via CEOAgent. The user query hits the CEOAgent, which analyzes intent and delegates to specialized agents (SearchAgent for RAG, GraphAgent for SQL hierarchy, SOPAgent for policies).

Important Limitation

This is specialized delegation/orchestration. It is NOT unrestricted autonomous tool-calling loops. Restricting the agents prevents infinite loops and catastrophic API token burn.

12. Knowledge Graph: Architectural Evolution

Initial Approach & Problem

I built the prototype using NetworkX, storing nodes in memory and saving to JSON. It was fast and easy. However, stateless deployment erased the JSON file, and concurrency locks made updating the file dangerous.

Decision & Implementation

I migrated the entire graph to SQL. I created kg_nodes and kg_edges tables. Relational queries now reconstruct the organizational hierarchy.

Trade-Off

I gave up native graph traversal algorithms (like PageRank) but gained extreme persistence, ACID compliance, and better suitability to stateless backend deployment.

13. Meeting Intelligence: Data Transformation

The Problem

Meeting transcripts are unstructured. Application code requires predictable structures to create database Tasks.

Implementation & Defensive Parsing

I prompt the LLM to output a JSON array of action items. However, LLMs hallucinate conversational filler. To bridge the gap between probabilistic AI and deterministic code, I built strict Regex extraction (`re.search(r"\{.*\}")`) to isolate the payload. The extracted tasks are saved to PostgreSQL and immediately pushed to the client via WebSockets.

14. Problems I Actually Encountered (Debugging)

Bug 1: Groq Model Deprecations (404s)

Symptom: The AI suddenly stopped responding entirely, throwing HTTP 500s.

Investigation: Groq had deprecated the specific llama3 model string I hardcoded.

Fix & Why: I rewrote `llm_client.py` to catch 404 errors, dynamically query Groq's `/models` endpoint for active models, and automatically retry. This decoupling makes the system highly resilient.

Bug 2: Timezone Mismatches

Symptom: The frontend Analytics page crashed entirely.

Investigation: `asyncpg` returns timezone-aware datetimes from Postgres. Python's `sorted()` function crashed when comparing these against naive datetimes generated locally.

Fix: I implemented a strict UTC normalization utility in `analytics.py`. All dates are converted to UTC before any sorting occurs.

15. Security Layers

Request -> JWT Authentication -> RBAC -> ABAC (Dept Isolation) -> PII Redaction -> Vector Storage

Security was implemented at boundaries. JWT proves identity. RBAC restricts feature access. ABAC physically partitions document queries. PII redaction protects the LLM boundary.

16. Observability: Why AI Required It

A normal API returns HTTP 200 even if the AI hallucinates completely. Standard logs are useless. I built specific observability tables:

- `agent_logs`: Captures routing decisions to debug why CEOAgent chose the wrong sub-agent.
- `llm_usage`: Tracks token consumption to prevent silent budget drains.
- `ai_failures`: Captures LLM API timeouts.

17. Testing: What Did I Verify?

I built pytest suites targeting specific failure modes:

- ABAC Tests: Potential problem -> Cross-department data leakage. Test -> Unauthorized department queries must return 403.
- Ingestion Tests: Potential problem -> Invalid extraction crashing the thread. Test -> Pipeline isolation.

18. Deployment: When Design Met Infrastructure

Local Assumptions: Infinite disk space, persistent memory.
Deployment Reality: Render free tier provides ephemeral disks and strict memory limits.
Failure & Adaptation: As previously noted, NetworkX JSON and ChromaDB were wiped constantly. The engineering response was migrating everything?vectors and graph?into the managed Supabase PostgreSQL instance. Deployment changed the architecture.

19. Engineering Decisions That Changed the System

Problem | Decision | Trade-off | Current State
Graph Persistence | Migrate to SQL tables | Lost native graph traversal algorithms | SQL-backed kg_nodes
LLM 404 Errors | Dynamic fallback retry logic | Slightly slower response on first failure | Resilient LLM client
Keyword Retrieval | Add BM25 | Increased memory usage locally | Hybrid RAG pipeline
PII Leaks | Regex redaction | Misses obscure edge cases vs heavy NLP | Fast in-memory redaction

20. Current State Summary

Today, ProcessPilot AI is a deployed, React/FastAPI/Postgres stack featuring Hybrid RAG, a SQL-backed Knowledge Graph, specialized multi-agent routing, WebSockets, and PII protection.

21. What Is Not Complete (Limitations)

Feature | Status | Explanation
Reciprocal Rank Fusion (RRF) | NOT IMPLEMENTED | Planned improvement for Hybrid retrieval scores.
Autonomous Tool Loops | NOT IMPLEMENTED | System uses controlled routing, not unconstrained tools.
Formal SOC2 / GDPR | NOT VERIFIED | No formal certification evidence.
ChromaDB | LEGACY | Configured in early code but superseded by pgvector.

22. What I Would Do Differently

If I started again today, I would implement durable object storage (like AWS S3) immediately for document ingestion, rather than trying to optimize ephemeral disk processing. I would also integrate RAGAS early in development to objectively measure retrieval quality, rather than relying on manual query testing.

23. Final Lessons

- Architecture must respond to constraints: The migration from NetworkX to SQL taught me that deployment environments dictate design.
- Security must be designed into the data flow: Adding PII redaction after chunks are stored is impossible.
- Observability is critical for probabilistic systems: Catching model deprecations required tracking ai_failures, not just HTTP status codes.

I DID NOT START WITH:

"Which AI technologies should I use?"

I STARTED WITH:

"What problem am I trying to solve?"

Then:

"What information does the system need?"

"What architecture supports it?"

"What can fail?"

"How do I protect the data?"

"How do I observe failures?"

"How do I test it?"

"How does it behave under deployment constraints?"

"What should I change after learning from the implementation?"

ProcessPilot AI is the result of that iterative process: problem -> design -> implementation -> failure -> investigation -> adaptation -> deployment.